POC B — Native Audio Capture

Prove whether unprocessed iOS audio measurably improves pronunciation scoring

Owner: Harshan **Date:** 28 July 2026 · **Revision 2** — retargeted for React Native / Flutter **Status: Two gates.** Neither the spike nor the build begins before its gate clears.

---

1. What this POC answers

One question:

> **Does bypassing iOS voice processing measurably improve pronunciation scoring for our learners?**

Not "can we build a native app" — that is known. The question is whether the one capability native provides that web cannot is worth what it costs.

POC A is the control; this is the treatment. Fixture, scorer, backend, normalization and output format are identical and reused. **Only the capture path differs.** Any score difference between A and B is therefore attributable to capture, which is the entire point. If the two share anything less than that, the comparison is worthless.

2. What changed in this revision

The original plan assumed Capacitor, which wraps the existing React app and reuses its UI as-is. The chosen direction is React Native or Flutter, and **neither reuses web UI components.**

	Capacitor *(original)*	React Native	Flutter
Existing React UI reused	As-is	**No** — RN components are not web components	**No** — full rewrite
Language	TypeScript	TypeScript	Dart (new to the team)
Native code for measurement mode	~150 lines Swift	~150 lines Swift	~150 lines Swift
Effort to a testable POC	~8 days	**~3–4 weeks**	**~4–6 weeks**

The audio work is identical in all three. Every option reaches `AVAudioSession` through a platform channel or native module wrapping the same Swift. The framework choice does not touch the thing being tested — it only changes how much of Lingotran must be rebuilt before the test can run.

A caution worth stating plainly. "We know React, so React Native is easy" is half true. TypeScript, business logic, API clients, validation and types carry over. **Components do not.** `<div>` is not `<View>`, CSS is not StyleSheet, routing differs. The recording screen is rebuilt, not ported.

Consequence for the plan: a bare framework build costs 3–6 weeks before it can answer a question a 2-day spike can answer. This revision therefore inserts that spike as a hard gate.

---

3. Two gates

Gate 1 — POC A verdict

POC A outcome	Do with POC B
Sets separate cleanly on all platforms	**Cancel.** Web is sufficient; native buys nothing measurable.
Sets separate; iPhone consistently offset	**Spike only.** Run Phase 1 to size the gap; skip the build if calibration closes it.
Sets separate on desktop; unreliable on iPhone	**Proceed to Phase 1.**
Sets do not separate anywhere	**Cancel.** The scorer is the problem; capture is irrelevant.

Gate 2 — The audio spike

Phase 1 is a **two-day audio-only spike**: a bare app, a native module, measurement mode, record a WAV, run the three acoustic checks. No product surface, no UI, no navigation.

Spike result	Action
Native capture measurably differs from web on the same device	**Proceed to Phase 2** — the framework build is justified
Native capture measures the same as web	**Stop.** Two days spent instead of four weeks. Either iOS applies processing regardless of session mode, or the module is misconfigured — investigate before spending further.

Gate 2 exists because of the cost change. At 8 days, building first and measuring after was defensible. At 3–6 weeks, it is not.

---

4. Architecture

```
React Native / Flutter app
   │
   │  platform channel / native module
   ▼
NATIVE AUDIO MODULE
   │
   ├── iOS (Swift)                    ── the point of the whole POC
   │   AVAudioSession.setCategory(.record, mode: .measurement)
   │   setPreferredSampleRate(16000)
   │   AVAudioEngine.inputNode.installTap  →  Int16 PCM
   │
   └── Android (Kotlin)
       AudioRecord( AudioSource.UNPROCESSED )    ← check availability
       fallback: AudioSource.VOICE_RECOGNITION
       16 kHz mono PCM16
   │
   ▼
Int16 PCM  ──►  WAV
   │
   ▼
POST /api/v1/pronunciation      ← IDENTICAL endpoint to POC A
   │
   ▼
Same Node backend, same Azure call, same normalization, same schema
```

No backend work in this POC. Everything below the upload is reused verbatim from POC A.

---

5. Phase 1 — The audio spike (2 days)

Roles: **MOB** mobile/native · **QA** test · **LEAD** Harshan

ID	Task	Role	Days	Depends on
B1	Bare RN or Flutter app — one screen, one record button, no product code	MOB	0.5	Gate 1
B2	iOS native module — `AVAudioSession` record category with measurement mode, engine input tap, 16 kHz mono Int16, write WAV to disk	MOB	1	B1
B3	**Acoustic verification** — run the three `probe.html` measurements against native-captured audio: same device, same room, same operator as the web capture	QA	0.5	B2
B4	Spike verdict — does native capture measurably differ from web capture?	LEAD	0.25	B3

B3 acceptance — the integrity check

Measurement	Web capture (iPhone)	Native capture (iPhone)	Expected if the module works
Noise floor, 4 s silence	—	—	Native **higher** — real room tone, not gated silence
Dynamic range, quiet vs loud	—	—	Native **wider** — no gain compression
Fricative energy, sustained "sss"	—	—	Native **higher** in the 4–8 kHz band

If these do not differ, stop. Either iOS applies processing regardless of the session mode, or the module is misconfigured. Both need investigation before another day is spent, and neither is discovered by building more UI.

This is the single most important package in the POC. Do not accept an API call returning success as evidence. Measure the audio.

---

6. Phase 2 — The framework build

Only after Gate 2 clears.

ID	Task	Role	Days RN	Days Flutter	Depends on
B5	Project setup — navigation, state, API client, build config for both platforms	MOB	3	4	Gate 2
B6	Port business logic — API clients, types, validation, scoring models. TypeScript carries over for RN; Dart rewrite for Flutter.	MOB	3	6	B5
B7	**Rebuild the recording screen** — record button, level meter, prompt display, per-word and per-phoneme feedback. A rebuild in native components, not a port.	MOB	5	6	B5
B8	Wire the native audio module into the app, with the same VAD and SNR gate logic as POC A	MOB	2	3	B2, B7
B9	Android native module — `AudioRecord` with `UNPROCESSED`, runtime capability check, logged fallback	MOB	2	2	B5
B10	Upload path to the existing endpoint, with retry	MOB	1	1.5	B8
B11	Permission and lifecycle — mic permission, interruption from calls and Siri, backgrounding	MOB	2	2.5	B8
B12	Re-run B3 acoustic verification against the full app, not the spike	QA	0.5	0.5	B8
B13	Fixture run — the same 80 recordings, **same speakers**, on iPhone and iPad	QA	1.5	1.5	B12
B14	Android fixture run	QA	0.5	0.5	B9, B13
B15	A/B analysis — native vs web, per device, per set	LEAD	1.5	1.5	B13, A17
B16	Production cost and effort assessment	LEAD	1	1	B15

Phase 2 effort: ~23 person-days (React Native) · ~30 person-days (Flutter) **Calendar:** 3–4 weeks RN · 4–6 weeks Flutter, allowing for ramp-up

---

7. React Native or Flutter

Consideration	React Native	Flutter
Team language continuity	**TypeScript** — the existing team reads and reviews from day one	Dart — new language, new toolchain, team split across two languages
Business logic reuse from Lingotran	Types, API clients, validation port over	Rewrite
UI component reuse	None	None
Rendering consistency and animation	Good	**Better** — matters for heavily gamified interfaces
Native module authoring	Swift / Kotlin, well documented	Swift / Kotlin via platform channels, equally documented
Proximity to existing team skills	Closer	Further

For Lingotran, React Native. TypeScript continuity is decisive — the same people work across web and native without a second language. Flutter's rendering advantage is real, but matters most for animation-heavy interfaces; if the product is largely prompts, forms and audio interactions, that advantage is small against the cost of carrying a second language.

---

8. The decision this POC produces

Result	Meaning	Action
Native scores match web scores	iOS processing was not degrading anything	Ship on web. The spike alone saved a month.
Native improves iPhone scores, closing the gap to desktop	Processing was the cause; native fixes it	Productionise the native path, with a real number behind the decision
Native improves scores, but calibration achieves the same	The gap is a correctable offset	Ship on web with calibration; native is not worth its cost
Native does not improve scores and iPhone remains unreliable	Something else is wrong	Stop. The problem is not capture — investigate the scorer, fixture, or reference-text handling.

Two of four end with "ship on web." That is the expected shape of a well-designed POC: most should conclude that the expensive option was unnecessary.

---

9. Risks

Risk	Mitigation
Module appears to work but processing is still applied	B3 and B12 exist for this. Never trust an API return value; measure the audio.
Building the framework app before knowing whether native audio differs	**Gate 2.** The single largest cost risk in this plan.
"We know React, RN will be quick"	Budget the rebuild honestly. Components do not port. B7 is five days, not one.
Different speakers record the native fixture than the web fixture	**Invalidates the comparison.** Same speakers, same words, same session where possible.
Dart ramp-up if Flutter is chosen	Add a week. It is not included in the estimates above.
iOS measurement mode alters audio routing	Test the full prompt-then-record loop, not capture in isolation
Android `UNPROCESSED` unsupported on test devices	Runtime capability check with logged fallback. Android is the secondary target.
Scope creep — the POC becomes the product	Only the recording flow is in scope. No auth, lessons, progress, or navigation beyond one screen.

10. Out of scope

App Store submission. Push notifications. Offline mode. Deep linking. In-app purchase. Any Lingotran feature other than the recording flow. Production release process. Android UI polish.

11. Deliverables

Phase 1 — spike app, iOS native audio module, B3 acoustic verification report, spike verdict.

Phase 2 — RN or Flutter app with the recording flow, iOS and Android audio modules, fixture results in the POC A schema, A/B analysis, production cost estimate.

12. Results template

Device	Path	Set A median	Set B median	Separation	Δ vs web
iPhone	Web (POC A)	—	—	—	—
iPhone	**Native**	—	—	—	—
iPad	Web	—	—	—	—
iPad	**Native**	—	—	—	—
Android	Web	—	—	—	—
Android	**Native**	—	—	—	—
Mac Chrome	Web (control)	—	—	—	—

The number that decides it: iPhone native separation minus iPhone web separation. If that difference is small, native is not worth building. If it is large and calibration cannot close it, it is.

---

13. What this POC does not tell us

- Whether an app is right for **business** reasons — retention, push notifications, store presence, offline use. Those arguments are real and entirely separate from this experiment.
- Whether the scorer is good enough in absolute terms. POC A answers that.
- Anything about classroom noise, network conditions, or accent bias in the model. None of those change with platform.
- The true cost of productionising beyond B16's estimate.

A native app may still be the right decision for reasons this POC does not measure. It should simply not be justified by an audio argument that this POC has not supported.